Software Measurement and Effort Estimation

Lecture 8

Agenda

- Software Measurement
- Effort Estimation
- COSMIC

Basic Concepts

Cost Versus Effort

- Although different, the terms “cost” and “effort” are often used as synonyms in the software project management area.
- The Webster dictionary defines
  - cost as “the amount or equivalent paid or charged for something” and
  - effort as “the total work done to achieve a particular end”.
- Effort refers to staff time spent on performing activities aimed at providing certain products or services (e.g., person-hours, person-days, person-months, etc.).
- Project cost includes, but is not limited to, project effort.

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Estimation Versus Prediction Versus Planning

- Estimation
  - "the act of judging tentatively or approximately the value, worth, or significance of something"
- Prediction
  - "the act of declaring or indicate in advance; especially, foretelling on the basis of observation, experience, or scientific reason"
- Planning
  - "the act or process of making or carrying out plans; specifically, the establishment of goals, policies, and procedures for a social or economic unit"

© Dra. Villavicencio, © Dr. Mera-Gómez

Estimation Versus Prediction

- Both estimation and prediction contain an element of uncertainty
- The first refers to approximating an actual state, whereas the latter refers to a future state.
- Simplifying, we may define prediction as estimating in advance.

Prediction Versus Planning

- Prediction refers to an unbiased, analytical process of approximating a future state.
- Planning, on the other hand, refers to a biased process of establishing goals with respect to the future state.
- The goal of prediction is to accurately foretell resources (such as effort) required to provide project outcomes.
- The goal of effort planning is, on the other hand, to plan the project in such a way that the project goals are achieved.
- In other words, we plan means within a project to achieve a specific project's end.

Importance of Estimation

Importance of Software Estimation

- Why should you care about estimation and measurement in software?
  - Because the ability to deliver software of required functionality and quality within cost and schedule is critical to all software customers.
- When we look at the track record for the software industry, although it has improved over the last years, a disappointing picture still emerges (Laird and Brennan, 2006):
  - A full 23% of all software projects are canceled before completion.
  - Of those projects completed, only 28% were delivered on time, within budget, and with all originally specified features.
  - The average software project overran the budget by 45%.

Importance of Software Estimation

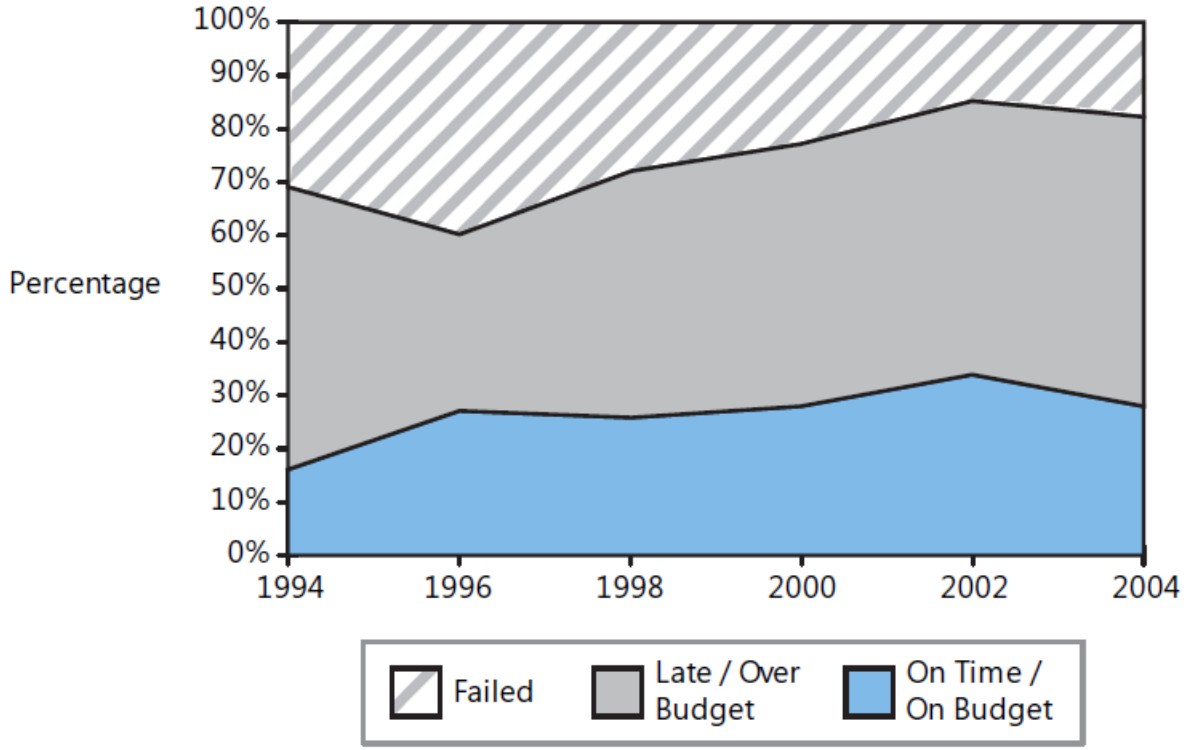


Figure 3-2 Project outcomes reported in The Standish Group's Chaos report have fluctuated year to year. About three quarters of all software projects are delivered late or fail outright.

Importance of Software Estimation

- Only 39% of software projects are successful (B13.ai, 2023):
  - 52.7% of projects exceed their initial estimates up to a 189%.
  - Only 16.2% of software projects are completed on time and on budget.
  - Quality is a pivotal concern. Only 52% of projects meet post-release benchmarks.
  - Poor requirements are the leading cause of project failure at 39%.
  - 33% of the projects fail due to lack of a formal software development process.
  - Delays are rampant, with 20% of initiatives falling behind schedule.
  - An astounding 78% encounter scope creep.
  - 45% of features in a software project are never used resulting in wasted investment.

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Principles of Effort and Cost Estimation

Effort Estimation in Context

- The project scope triangle represents a trade-off between project outputs and inputs.
- The three competing objectives are:
  - software functionality,
  - software quality, and
  - resources needed to accomplish the former two.
- Everybody wants things good, wants them fast, and wants them cheap. Everyone knows you can actually achieve any two of the three.
  - Software engineering is not different.
  - People expect software of high quality, available quickly, and at low budget.

Project Equilibrium Triangles

- The resources triangle represents a trade-off between project resources, namely, staffing, duration, and effort.

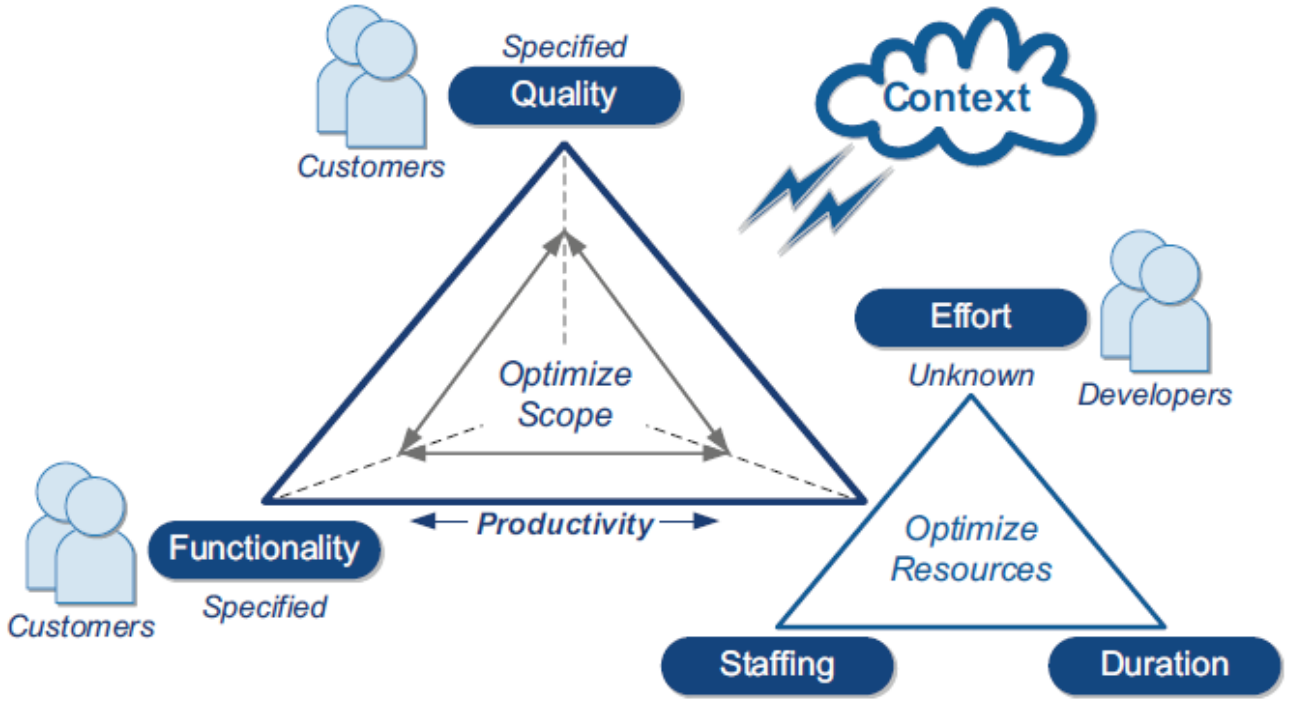


Fig. 2.1 Project equilibrium triangles

Project Equilibrium Triangles

- Staffing refers to the number of persons required to complete a certain project work, task, or activity.
- Duration refers to the total elapsed time needed to complete a particular work, including nonproductive time such as breaks, holidays, and sick leave.
- Effort is a combination of person and time. It refers to the amount of fully productive work time one person would need to complete a certain work.
- Budget refers to the amount of money available for project realization.

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Estimate Uncertainties

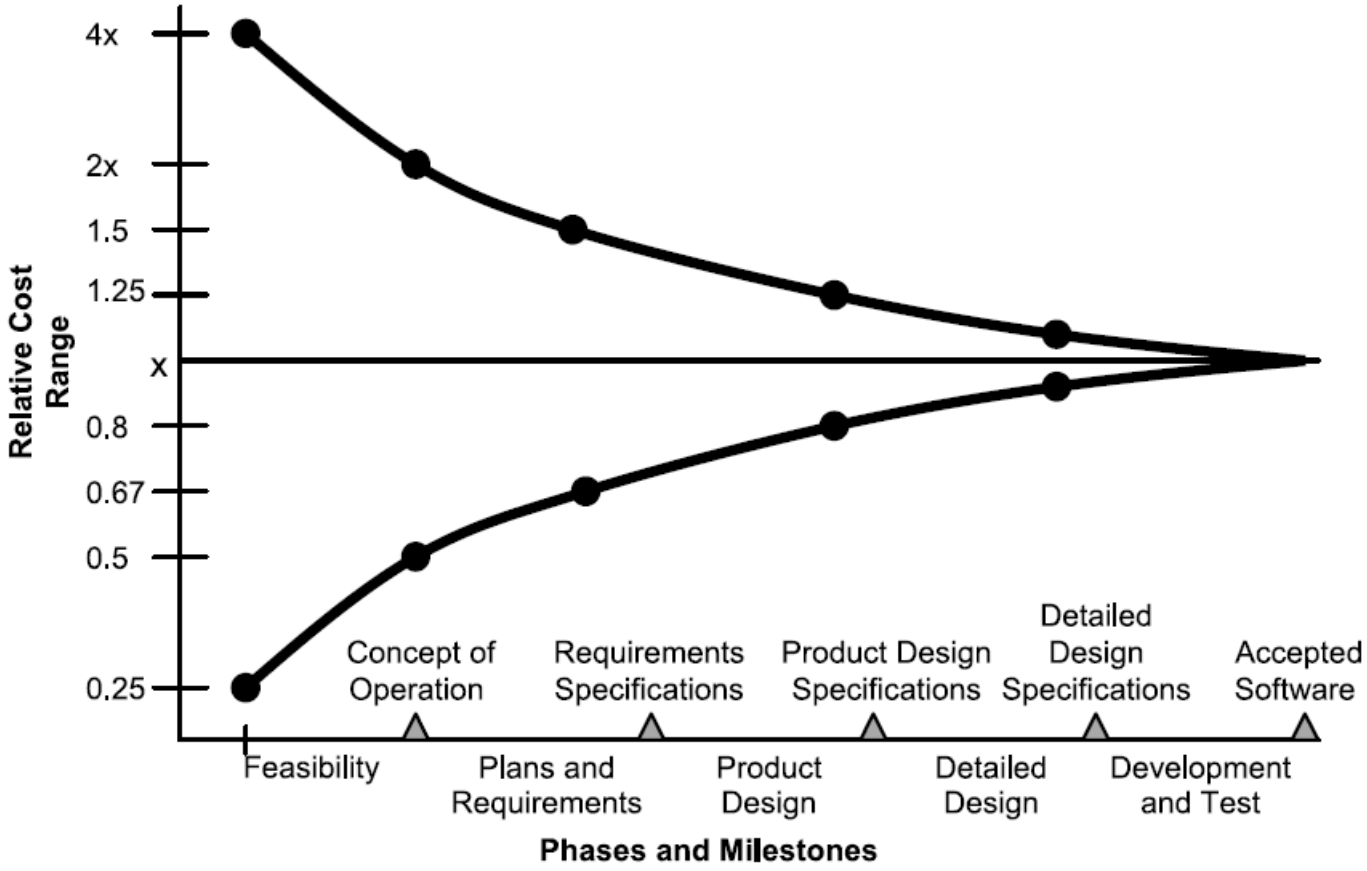


Figure 6.22. *The cone of uncertainty.*

Estimates as Probability Statements

- Software estimates are routinely presented as single-point numbers, such as “This project will take 14 weeks.” Such simplistic single-point estimates are meaningless because they don’t include any indication of the probability associated with the single point.

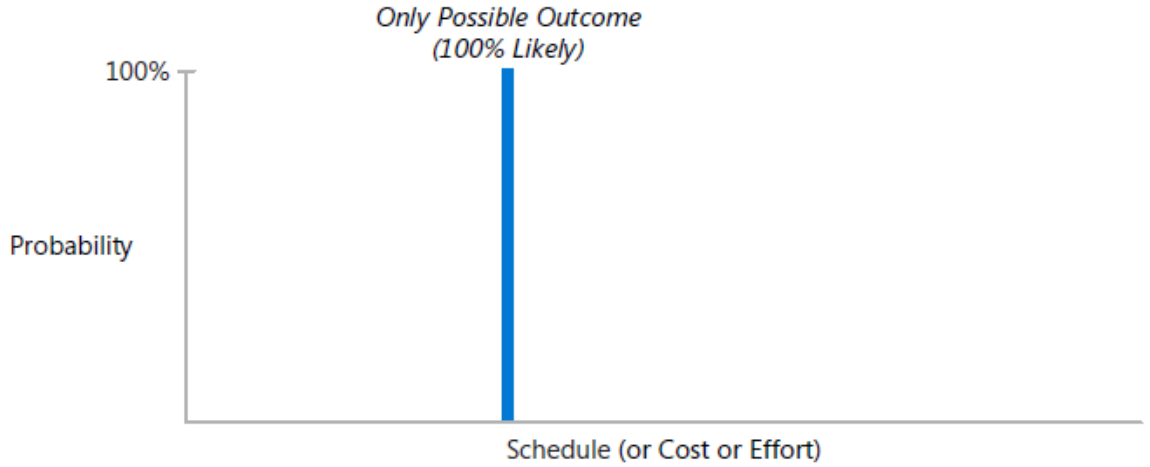


Figure 1-1 Single-point estimates assume 100% probability of the actual outcome equaling the planned outcome. This isn’t realistic.

Estimates as Probability Statements

- When you see a single-point “estimate,” ask whether the number is an estimate or whether it’s really a target.
- The vertical dashed line shows the “nominal” outcome, which is also the “50/50” outcome—there’s a 50% chance that the project will finish better and a 50% chance that it will finish worse. Statistically, this is known as the “median” outcome.

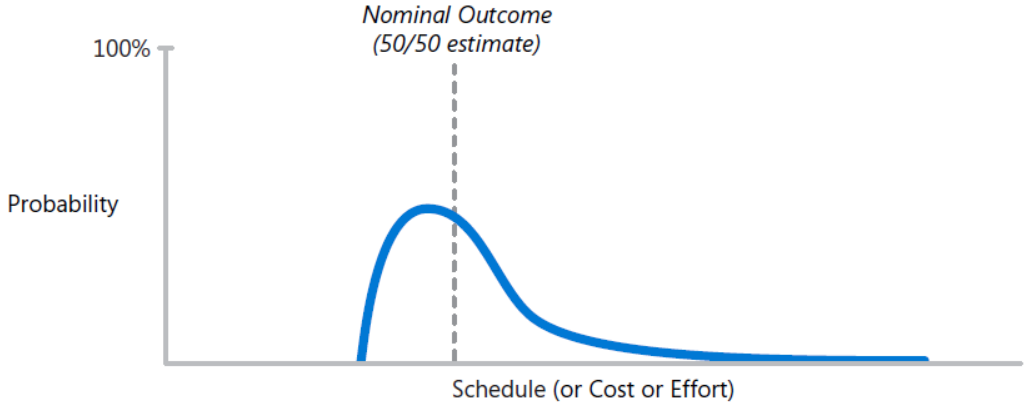


Figure 1-3 An accurate depiction of possible software project outcomes. There is a limit to how well a project can go but no limit to how many problems can occur.

Estimates as Probability Statements

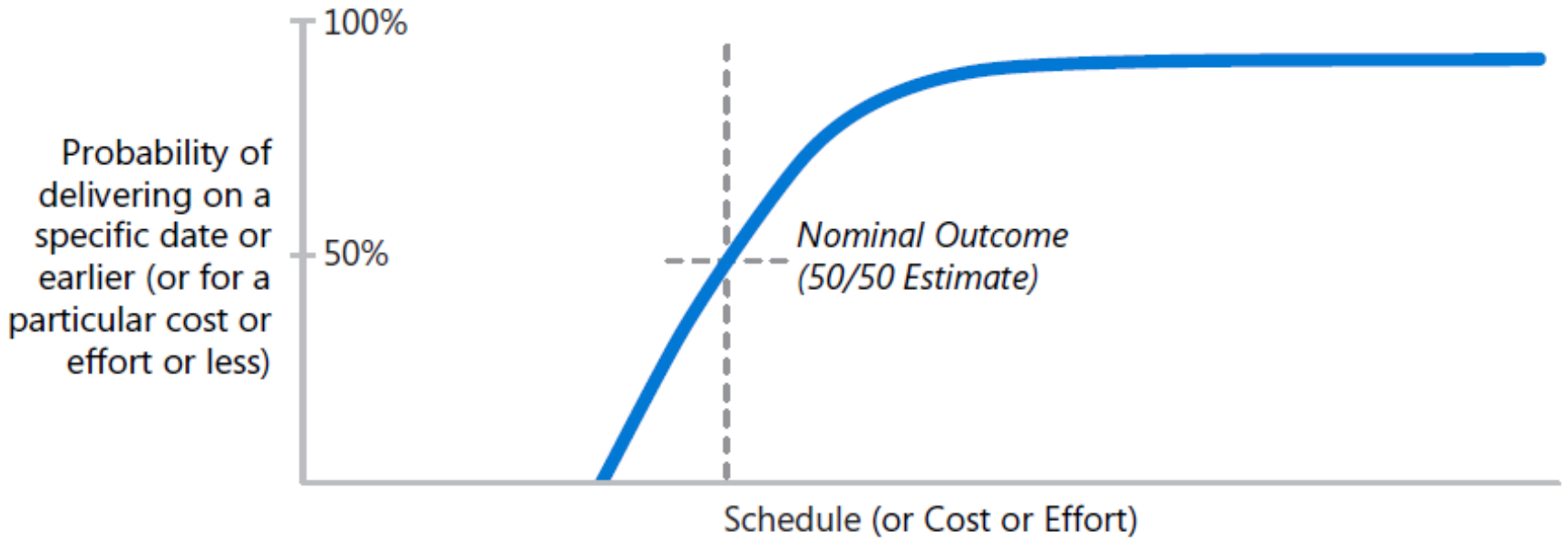


Figure 1-4 The probability of a software project delivering on or before a particular date (or less than or equal to a specific cost or level of effort).

Measures of Estimation Uncertainty

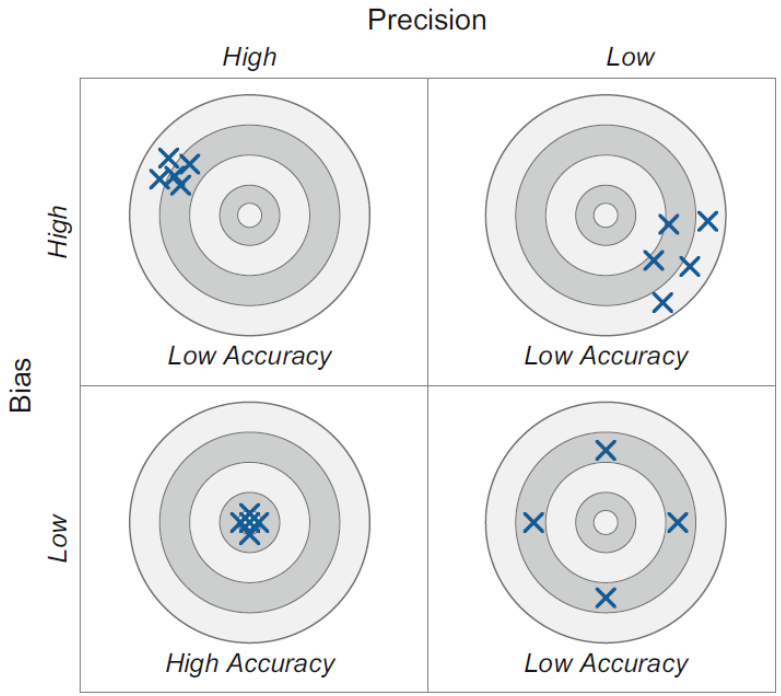


Fig. 4.3 Estimation accuracy, precision, and bias

- **Precision** refers to the degree to which several estimates are very close to each other. It is an indicator of the scatter in the data. The less the scatter, the higher is the precision.
- **Bias** refers to a systematic—constant—error in estimates and is determined as the difference between the average of estimates and the actual, true value.
- **Accuracy** is a function of bias and precision and describes the nearness of estimates to the true value. Highly accurate prediction methods will provide estimates very close to the actual, true value.

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Is It Better to Overestimate or Underestimate?

- Arguments Against Overestimation
  - if a project is overestimated, Parkinson's Law will kick in—the adage that work will expand to fill the time allotted for its completion. Deadlines can cause prompt people to fill their time with trivial matters.
  - Another concern is Goldratt's "Student Syndrome". If developers are given too much time, they'll procrastinate until late in the project, at which point they'll rush to complete their work, and they probably won't finish the project on time.

Is It Better to Overestimate or Underestimate?

- Arguments Against Underestimation
  - Reduced effectiveness of project plans
  - Statistically reduced chance of on-time completion: Developers typically estimate 20% to 30% lower than their actual effort
  - Poor technical foundation leads to worse-than-nominal results: A low estimate can cause you to spend too little time on upstream activities such as requirements and design
  - Destructive late-project dynamics make the project worse than nominal: Once a project gets into “late” status, project teams engage in numerous activities that they don’t need to engage in during an “on-time” project

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Is It Better to Overestimate or Underestimate?

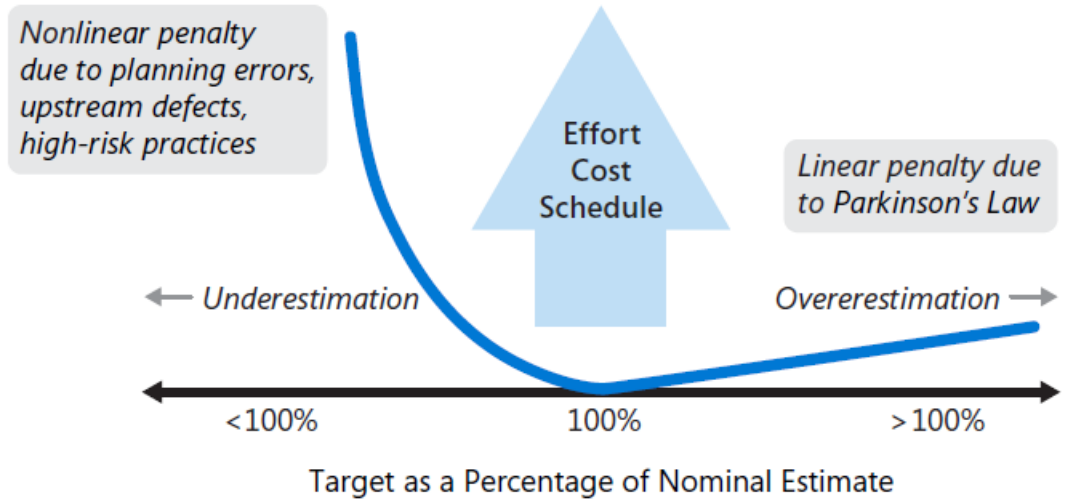


Figure 3-1 The penalties for underestimation are more severe than the penalties for overestimation, so, if you can't estimate with complete accuracy, try to err on the side of overestimation rather than underestimation.

How Late Are the Late Projects?

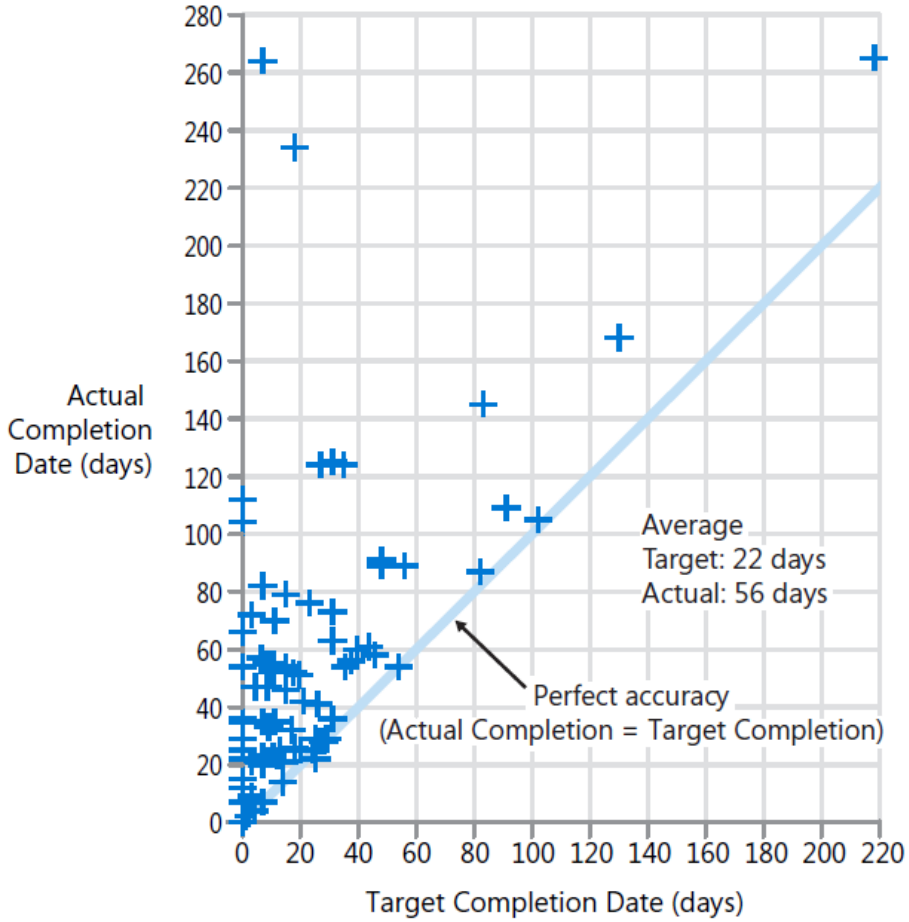


Figure 3-3 Estimation results from one organization. General industry data suggests that this company's estimates being about 100% low is typical. Data used by permission.

Benefits of Accurate Estimates

- Improved status visibility
  - If the planned progress is realistic (that is, based on accurate estimates), it's possible to track progress according to plan.
- Higher quality
  - Accurate estimates help avoid schedule-stress-related quality problems. About 40% of all software errors have been found to be caused by stress
- Better coordination with non software functions
  - Software projects usually need to coordinate with other business functions, including testing, document writing, marketing campaigns, sales staff training, financial projections, software support training, and so on.

Benefits of Accurate Estimates

- Better budgeting
  - Although it is almost too obvious to state, accurate estimates support accurate budgets.
- Increased credibility for the development team
  - A project team that holds its ground and insists on an accurate estimate will improve its credibility within its organization.
- Early risk information
  - The detection of a mismatch between the project goal and the project estimate should be interpreted as incredibly useful, incredibly rare, early-in-the-project risk information. The mismatch indicates a substantial chance that the project will fail to meet its business objective.

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Where Does Estimation Error Come From?

Where Does Estimation Error Come From?

- Inaccurate information about the project being estimated
- Inaccurate information about the capabilities of the organization that will perform the project
- Too much chaos in the project to support accurate estimation (that is, trying to estimate a moving target)
- Inaccuracies arising from the estimation process itself

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

The Cone of Uncertainty

- Software development consists of making literally thousands of decisions about all the feature-related issues described in the previous section.
  - As you make a greater percentage of those decisions, you reduce the estimation uncertainty.

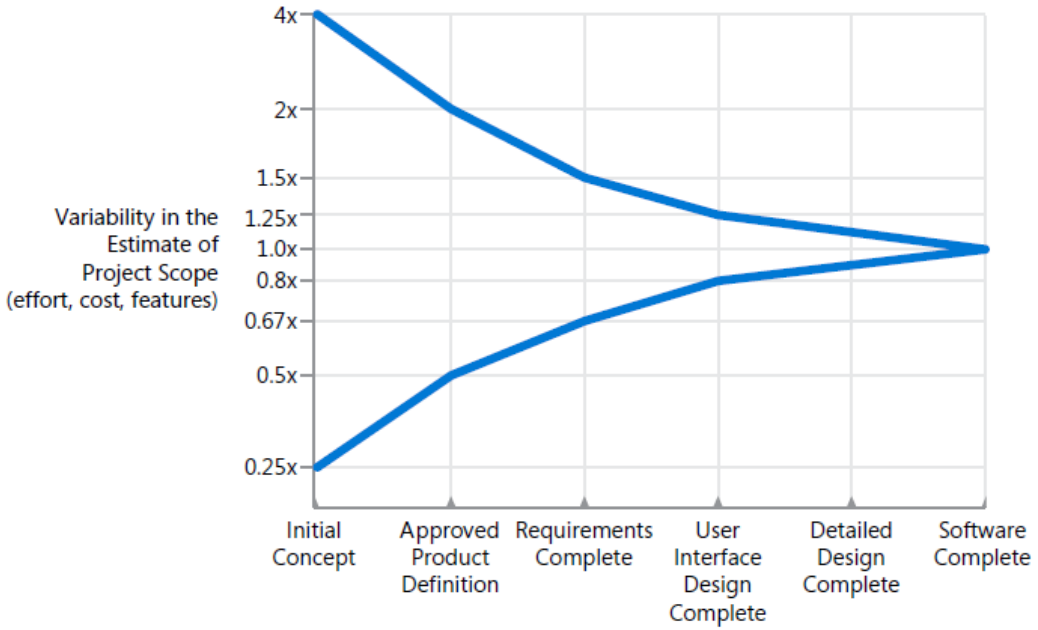


Figure 4-1 The Cone of Uncertainty based on common project milestones.

Chaotic Development Processes

- Common examples of project chaos include the following:
  - Requirements that weren't investigated very well in the first place
  - Lack of end-user involvement in requirements validation
  - Poor designs that lead to numerous errors in the code
  - Poor coding practices that give rise to extensive bug fixing
  - Inexperienced personnel
  - Incomplete or unskilled project planning
  - Prima donna team members
  - Abandoning planning under pressure
  - Developer gold-plating
  - Lack of automated source code control

Unstable Requirements

- The first challenge is that unstable requirements represent one specific flavor of project chaos.
  - If requirements cannot be stabilized, the Cone of Uncertainty can't be narrowed, and estimation variability will remain high through the end of the project.
- The second challenge is that requirements changes are often not tracked and the project is often not re-estimated when it should be.
  - In a well-run project, an initial set of requirements will be baselined, and cost and schedule will be estimated from that baselined set of requirements.
  - If your environment doesn't allow you to stabilize requirements, consider alternative development approaches that are designed to work in high-volatility environments, such as short iterations, Scrum, Extreme Programming, DSDM (Dynamic Systems Development Method), time box development, and so on.

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Omitted Activities

Table 4-3 **Software-Development Activities Commonly Missing from Software Estimates**

Ramp-up time for new team members	Technical support of existing systems during the project
Mentoring of new team members	Maintenance work on previous systems during the project
Management coordination/manager meetings	Defect-correction work
Cutover/deployment	Performance tuning
Data conversion	Learning new development tools
Installation	Administrative work related to defect tracking
Customization	Coordination with test (for developers)
Requirements clarifications	Coordination with developers (for test)
Maintaining the revision control system	Answering questions from quality assurance
Supporting the build	Input to user documentation and review of user documentation
Maintaining the scripts required to run the daily build	Review of technical documentation
Maintaining the automated smoke test used in conjunction with the daily build	Demonstrating software to customers or users
Installation of test builds at user location(s)	Demonstrating software at trade shows
Creation of test data	Demonstrating the software or prototypes of the software to upper management, clients, and end users
Management of beta test program	Interacting with clients or end users; supporting beta installations at client locations
Participation in technical reviews	Reviewing plans, estimates, architecture, detailed designs, stage plans, code, test cases, and so on
Integration work	
Processing change requests	
Attendance at change-control/triage meetings	
Coordinating with subcontractors	

Omitted Activities

Table 4-4 **Non-Software-Development Activities Commonly Missing from Software Estimates**

Vacations	Company meetings
Holidays	Department meetings
Sick days	Setting up new workstations
Training	Installing new versions of tools on workstations
Weekends	Troubleshooting hardware and software problems

Unfounded Optimism

- A study of about 100 schedule estimates within the U.S. Department of Defense found a consistent “fantasy factor” of about 1.33 (Boehm 1981). Common variations on this optimism theme include the following:
  - We’ll be more productive on this project than we were on the last project.
  - A lot of things went wrong on the last project. Not so many things will go wrong on this project.
  - We started the project slowly and were climbing a steep learning curve. We learned a lot of lessons the hard way, but all the lessons we learned will allow us to finish the project much faster than we started it.

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Other Sources of Error

- Unfamiliar business area
- Unfamiliar technology area
- Incorrect conversion from estimated time to project time (for example, assuming the project team will focus on the project eight hours per day, five days per week)
- Misunderstanding of statistical concepts (especially adding together a set of “best case” estimates or a set of “worst case” estimates)
- Having an accurate size estimate, but introducing errors when converting the size estimate to an effort estimate

Other Sources of Error

- Budgeting processes that undermine effective estimation (especially those that require final budget approval in the wide part of the Cone of Uncertainty)
- Having accurate size and effort estimates, but introducing errors when converting those to a schedule estimate
- Overstated savings from new development tools or methods
- Simplification of the estimate as it's reported up layers of management, fed into the budgeting process, and so on

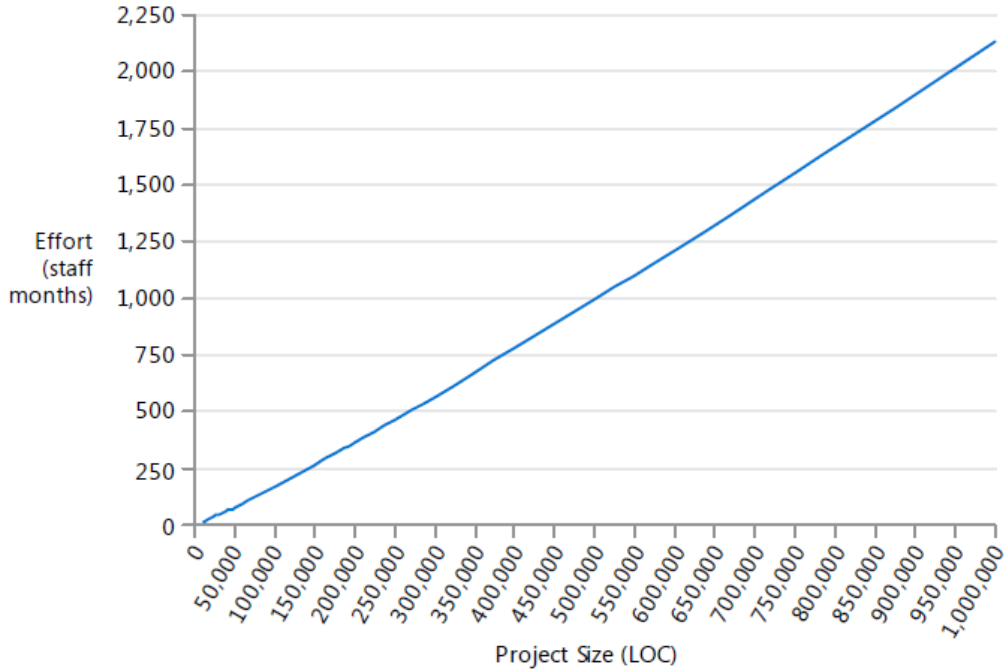
Estimate Influences

Estimate Influences

- Understanding the influences on a software project helps improve estimation accuracy and helps improve understanding of software project dynamics overall.
  - Project Size
  - Kind of Software Being Developed
  - Personnel Factors
  - Programming Language

Project Size

- The largest driver in a software estimate is the size of the software being built, because there is more variation in the size than in any other factor.



Source: Computed using data from the Cocomo II estimation model, assuming nominal diseconomy of scale (Boehm, et al 2000).

Figure 5-1 Growth in effort for a typical business-systems project. The specific numbers are meaningful only for the average business-systems project. The general dynamic applies to software projects of all kinds.

Diseconomies of Scale

- People naturally assume that a system that is 10 times as large as another system will require something like 10 times as much effort to build.
  - But the effort for a 1,000,000-LOC system is more than 10 times as large as the effort for a 100,000-LOC system.

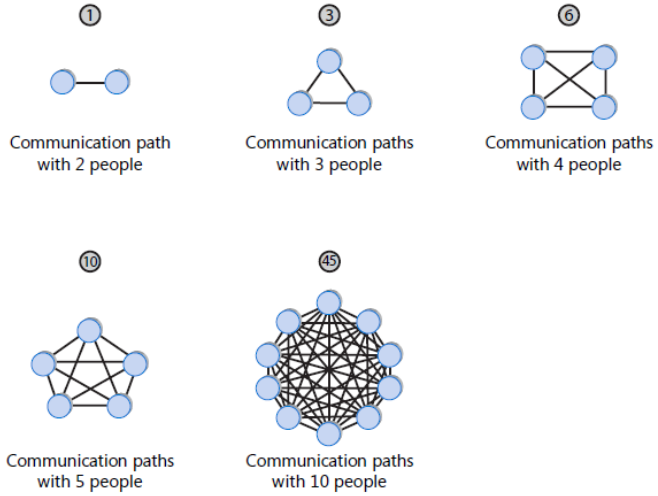
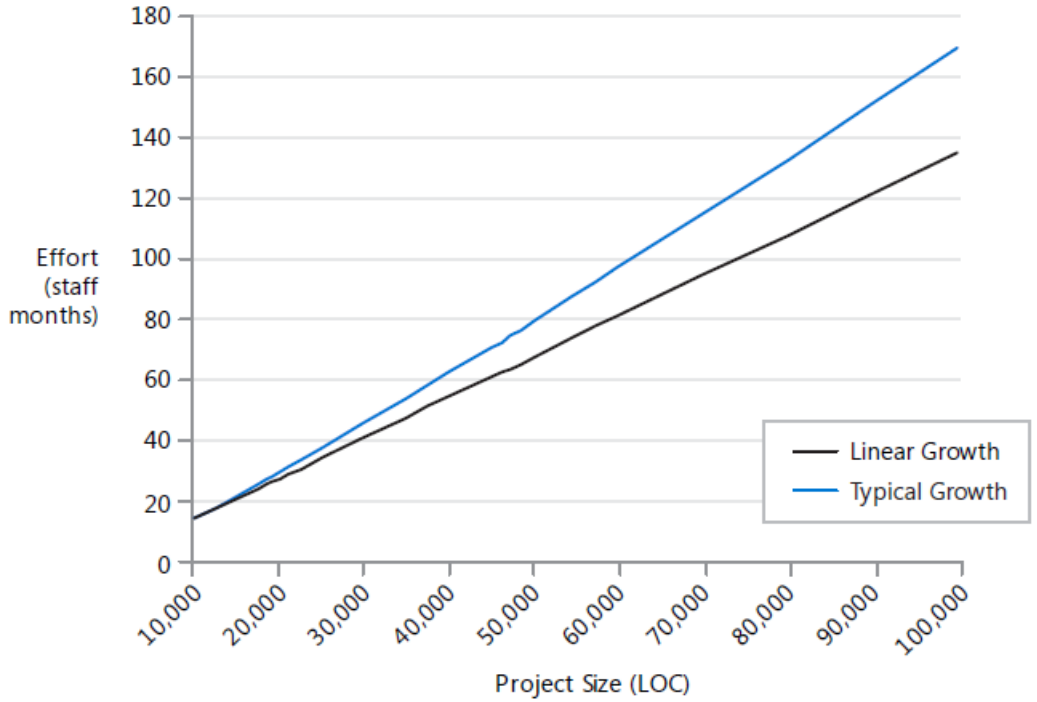


Figure 5-2 The number of communication paths on a project increases proportionally to the square of the number of people on the team.

Diseconomies of Scale



Source: Computed using data from the Cocomo II estimation model, assuming nominal diseconomy of scale (Boehm, et al 2000).

Figure 5-3 Diseconomy of scale for a typical business-systems project ranging from 10,000 to 100,000 lines of code.

Kind of Software Being Developed

Table 5-2 Productivity Rates for Common Project Types

	LOC/Staff Month Low-High (Nominal)		
Kind of Software	**10,000-LOC Project**	**100,000-LOC Project**	**250,000-LOC Project**
Avionics	100–1,000 **(200)**	20–300 **(50)**	20–200 **(40)**
Business Systems	800–18,000 **(3,000)**	200–7,000 **(600)**	100–5,000 **(500)**
Command and Control	200–3,000 **(500)**	50–600 **(100)**	40–500 **(80)**
Embedded Systems	100–2,000 **(300)**	30–500 **(70)**	20–400 **(60)**
Internet Systems (public)	600–10,000 **(1,500)**	100–2,000 **(300)**	100–1,500 **(200)**
Intranet Systems (internal)	1,500–18,000 **(4,000)**	300–7,000 **(800)**	200–5,000 **(600)**
Microcode	100–800 **(200)**	20–200 **(40)**	20–100 **(30)**
Process Control	500–5,000 **(1,000)**	100–1,000 **(300)**	80–900 **(200)**
Real-Time	100–1,500 **(200)**	20–300 **(50)**	20–300 **(40)**
Scientific Systems/ Engineering Research	500–7,500 **(1,000)**	100–1,500 **(300)**	80–1,000 **(200)**
Shrink wrap/Packaged Software	400–5,000 **(1,000)**	100–1,000 **(200)**	70–800 **(200)**
Systems Software/Drivers	200–5,000 **(600)**	50–1,000 **(100)**	40–800 **(90)**
Telecommunications	200–3,000 **(600)**	50–600 **(100)**	40–500 **(90)**

Source: Adapted and extended from *Measures for Excellence* (Putnam and Meyers 1992), *Industrial Strength Software* (Putnam and Meyers 1997), and *Five Core Metrics* (Putnam and Meyers 2003).

Personnel Factors

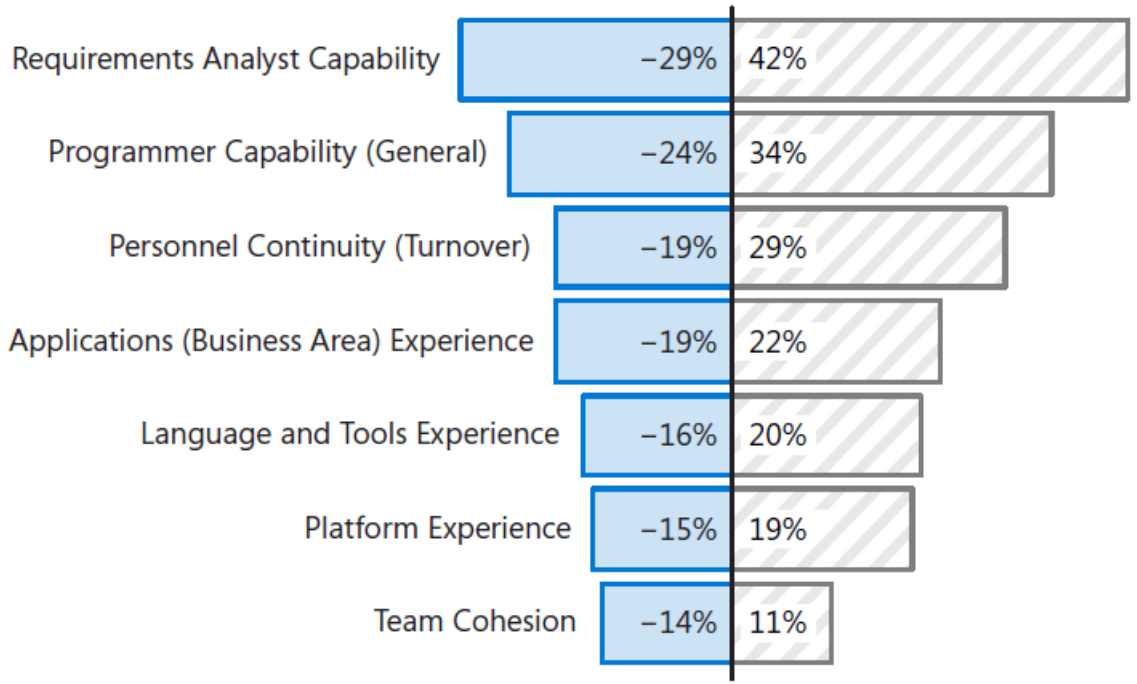


Figure 5-6 Effect of personnel factors on project effort. Depending on the strength or weakness in each factor, the project results can vary by the amount indicated—that is, a project with the worst requirements analysts would require 42% more effort than nominal, whereas a project with the best analysts would require 29% less effort than nominal.

Programming Language

- Some languages generate more functionality per line of code than others.
  - The Table shows the amount of functionality that several languages produce relative to the C programming language.

Table 5-3 Ratio of High-Level-Language Statements to Equivalent C Code

Language	Level Relative to C
C	1 to 1
C#	1 to 2.5
C++	1 to 2.5
Cobol	1 to 1.5
Fortran 95	1 to 2
Java	1 to 2.5
Macro Assembly	2 to 1

Table 5-3 Ratio of High-Level-Language Statements to Equivalent C Code

Language	Level Relative to C
Perl	1 to 6
Smalltalk	1 to 6
SQL	1 to 10
Visual Basic	1 to 4.5

Source: Adapted from *Estimating Software Costs* (Jones 1998) and *Software Cost Estimation with Cocomo II* (Boehm 2000).

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Measuring the Size of Software

Measuring the Size of Software

- Size is one of the most basic attributes of software. Some of the key questions people want to know about a project involve size. For example:
  - How big is the software? How big is it compared to other projects?
  - How much effort is it going to take to build the software?
  - How does our quality compare to other projects?
  - How productive are we compared to other projects?

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Physical Measurements of Software

Physical Measurements of Software

- The traditional measure for software size is the number of lines of code (LOC).
- It is a simple measure, but it easily encourages undesirable behavior when used inappropriately. If productivity is measured based on LOC/programmer day, the more lines you write, whether or not they do something useful, the more productive you are.
- It is always lacking in representing the size of the functionality of a system. Consequently, we need some other size measurements as well.

Measuring Functionality

Measuring Functionality

- Consider what strategy you might use to size a system, based on what it needs to do rather than how it does it internally.
  - It is a little difficult, isn't it?
- One possible way would be to count the inputs, outputs, interfaces, and databases in a system.
- That is the function point (FP) approach, once you add inquiries as well.
- Function Point Analysis (FPA) was invented as an indirect measure for the functional size of a system.

Function Points

Function Points

- The system's functionality is decomposed into components of inputs, outputs, external interface files (maintained by another system), internal data files (maintained by this system), and inquiries;
- each component's difficulty is rated as simple, average, or complex;

TABLE 4.3 Function Point Complexity Ratings

Component	Simple	Average	Complex
Inputs (I)	3	4	6
Outputs (O)	4	5	7
Data Files (F)	7	10	15
Interfaces (N)	5	7	10
Inquiries (Q)	3	4	6

Function Points

- a complexity rating is given to each component based on its type and difficulty;
- the Unadjusted Function Points (UFPs) are counted by summing all of the complexity ratings;
- a Value Adjustment Factor (VAF) is calculated based on the complexity of the overall system; and
- the Adjusted Function Point (AFP) count is calculated by multiplying VAF by the UFPs.

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Function Points

EXERCISE: You have a small program with four simple inputs, one data file (trivial), and two outputs, both of average complexity. How many unadjusted function points would this be?

Answer:

$$\text{Inputs} = 4 * 3 = 12$$
$$\text{Data File} = 1 * 7 = 7$$
$$\text{Ouputs} = 2 * 5 = 10$$
$$\text{Total UFPs} = 12 + 7 + 10 = 29$$

Function Points

- An additional adjustment is made based on the expected implementation difficulty of the system.
- Using the IFPUG standards, you calculate the VAF based on 14 General Systems Characteristics (GSCs), each of which is rated on a scale of 0 to 5, with 0 meaning no influence (or not present) and 5 meaning that characteristic has an extensive influence throughout the project
- The formula for adjusted function points is
  - AFPs = UFPs * (0.65 + 0.01 * VAF)

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Function Points

TABLE 4.4 General System Characteristics [10]

General System Characteristic (GSC)	Brief Description
1. Data Communications	How many communication facilities are there to aid in the transfer or exchange of information with the application of the system?
2. Distributed Data/Processing	How are distributed data and processing functions handled?
3. Performance Objectives	Are response time and throughput performance critical?
4. Heavily Used Configuration	How heavily used is the current hardware platform where the application will be executed?
5. Transaction Rate	Is the transaction rate high?
6. Online Data Entry	What percentage of the information is entered online?
7. End-User Efficiency	Is the application designed for end-user efficiency?
8. Online Update	How many data files are updated online?
9. Complex Processing	Is the internal processing complex?
10. Reusability	Is the application designed and developed to be reusable?
11. Conversion/Installation Ease	Are automated conversion and installation included in the system?
12. Operational Ease	How automated are operations such as backup, startup, and recovery?
13. Multiple Site Use	Is the application specifically designed, developed, and supported for multiple sites with multiple organizations?
14. Facilitate Change	Is the application specifically designed, developed, and supported to facilitate change and ease of use by the user?

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Function Points

- The VAF adjusts the function point count based on the "implementation difficulty" (i.e., the general system characteristics) of the system.
- Observe that for an "average" system, with all average values of 2.5 for the 14 GSCs, the VAF = 14 * 2.5 = 35,
  - such that the AFP = UFP * (0.65 + 0.35) = UFP, that is, the AFP equals 100% of the UFP.
  - The VAF can adjust the function point count by +/- 35% (if all of the GSCs are five or all are zero).

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Function Points

- EXERCISE: You have a small project that has 25 UFPs. You rate the GSCs of your system as trivial in all categories except complex processing and data communications, which have a high influence. All trivial factors are scored as 1.
- Complex processing and data communications both get scores of 5. How many AFPs do you have?

Answer:

$$\text{VAF} = 12 * 1 + 2 * 5 = 22$$
$$\text{AFPs} = \text{UFPs} * (0.65 + 0.01 * \text{VAF}) = 25 * (0.65 + 0.22) = 21.75$$

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

COSMIC

COSMIC

- Each data movement = 1 COSMIC Function Point (CFP)



COSMIC

- Entry
  - a data movement that moves a data group from a functional user across the boundary into the functional process where it is required.
- Exit
  - a data movement that moves a data group from a functional process across the boundary to the functional user that requires it.

COSMIC

- Read
  - a data movement that moves a data group from persistent storage into the functional process that requires it.
- Write
  - a data movement that moves a data group from inside a functional process to persistent storage.

Estimación en Ingeniería de Software

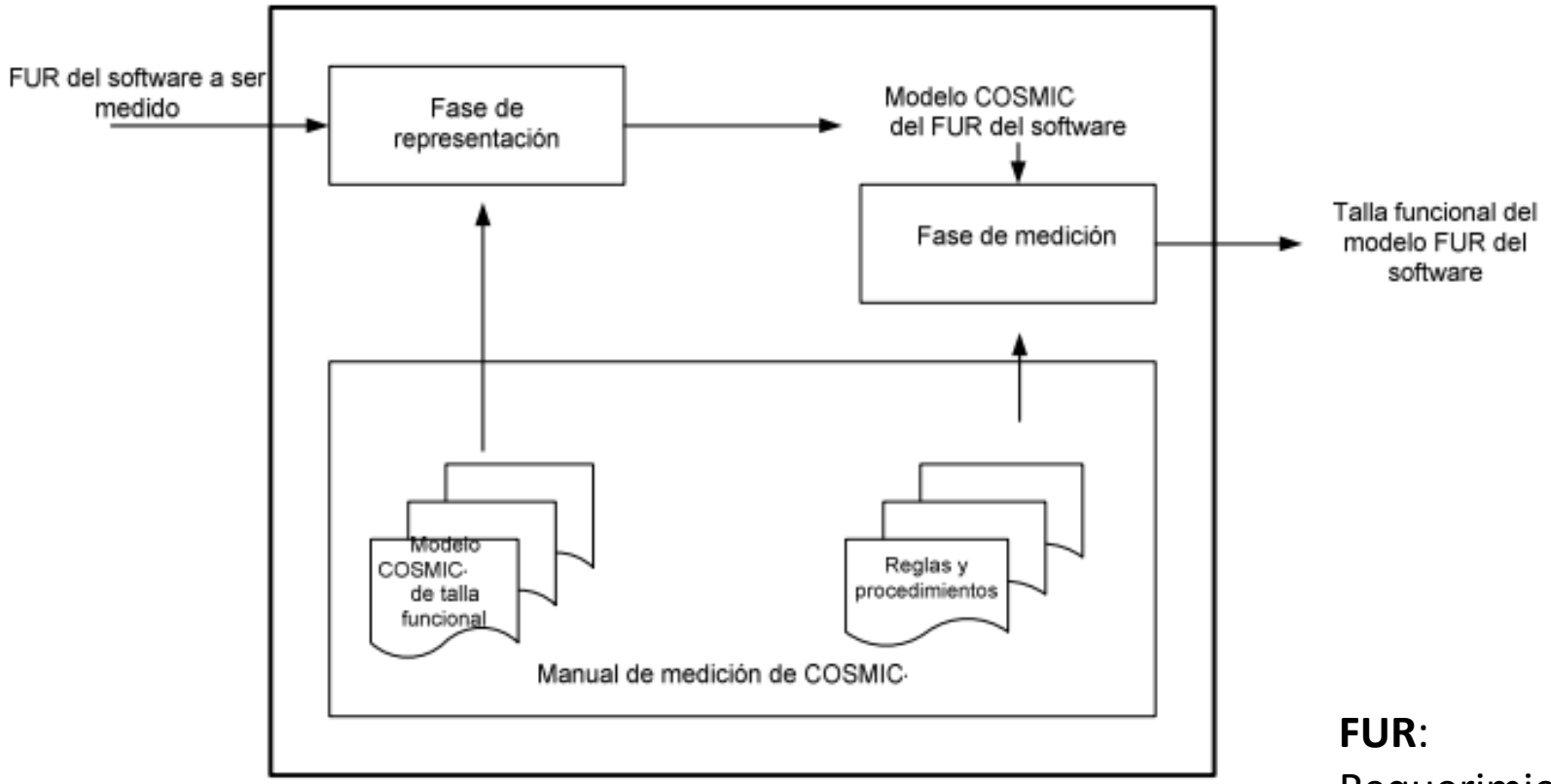
- Tiene por objetivo conocer el tamaño **aproximado** del sistema a desarrollar con el objeto de establecer el **costo**, la **duración** y los **recursos** necesarios para conseguirlo.
- Enfoques para estimación:
  - Valoraciones de expertos
  - Descomposición
  - Métodos evolucionarios y de prototipo
  - Métodos estadísticos
  - Uso de algoritmos: líneas de código [COCOMO], **requerimientos funcionales** [FPA, COSMIC], pantallas/reportes/ módulos, etc . [puntos de aplicación]

Estimación basada en funcionalidad

- **COSMIC** es un método para estimar el tamaño de un software basado en la especificación de sus requerimientos funcionales [ISO/IEC 14143/1:2007].
- Elaborado por Common Software Measurement International Consortium para:
  - Software de **aplicaciones de negocios** tales como banca, seguros, manufactura, distribución, etc.
  - Software en **tiempo real** tales como software embebido en dispositivos para control de máquinas. Ej.: motores de autos, aviones, electrodomésticos, sistemas operativos, etc.
  - **Híbridos** de ambos, como por ejemplo un sistema para reservaciones en aerolíneas.
- **No aplicable** para sistemas expertos, simuladores, procesamiento de imágenes, etc.

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

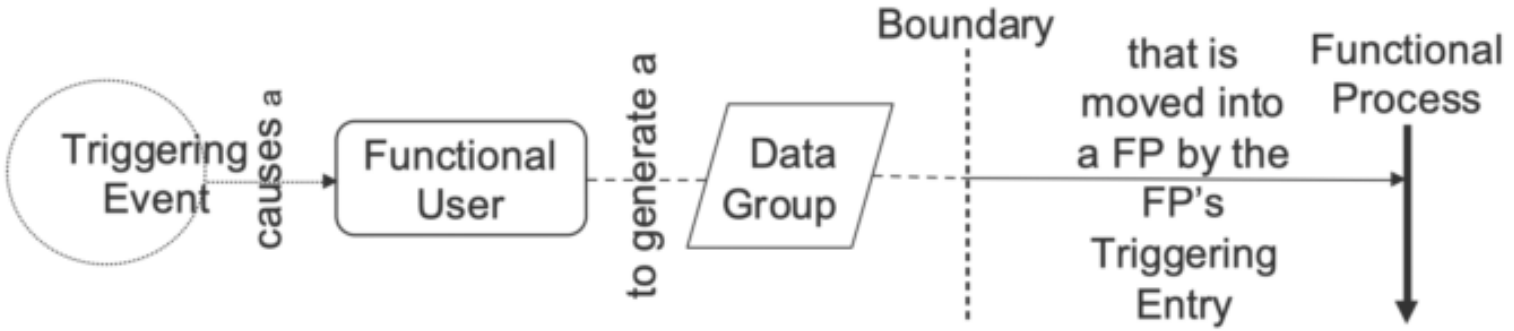
Método de estimación COSMIC



Modelo del método de estimación COSMIC
Fuente: http://www.itba.edu.ar

FUR: Requerimiento funcional del software

Método de estimación COSMIC



N.B. There can be many-to-many relationships at each stage along this chain, except that any one functional process may have only one triggering Entry

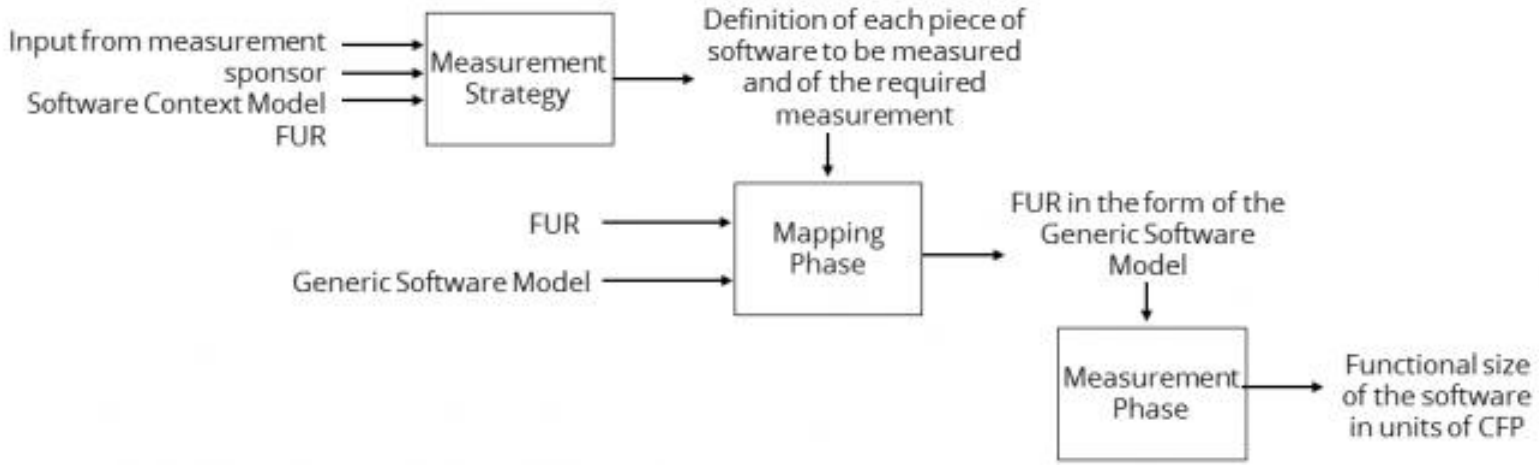
Copyright © COSMIC 2017

Fuente: https://www.scopemaster.com/introduction-to-cosmic-function-points/

Fases del método de estimación COSMIC

- **Fase I**: La fase de definición de la estrategia de medición
- **Fase II**: La fase de mapeo de FURs al modelo conceptual de COSMIC
- **Fase III**: La fase de medición de COSMIC
- **Fase IV**: La fase del reporte de la medición

Fases del método de estimación COSMIC



Fuente: https://cosmic-sizing.org/cosmic-fsm/

Fase I: La fase de definición de la estrategia de medición

- Se establece el **propósito de la medición**, es decir, las razones por las cuales se realiza la medición. Ej.:
  - **Estimar** el costo de un nuevo desarrollo de software.
  - **Comparar** el costo real con el costo estimado del software.
- Se define el **alcance de la medición**, es decir, el subconjunto de requerimientos funcionales incluidos en la estimación.

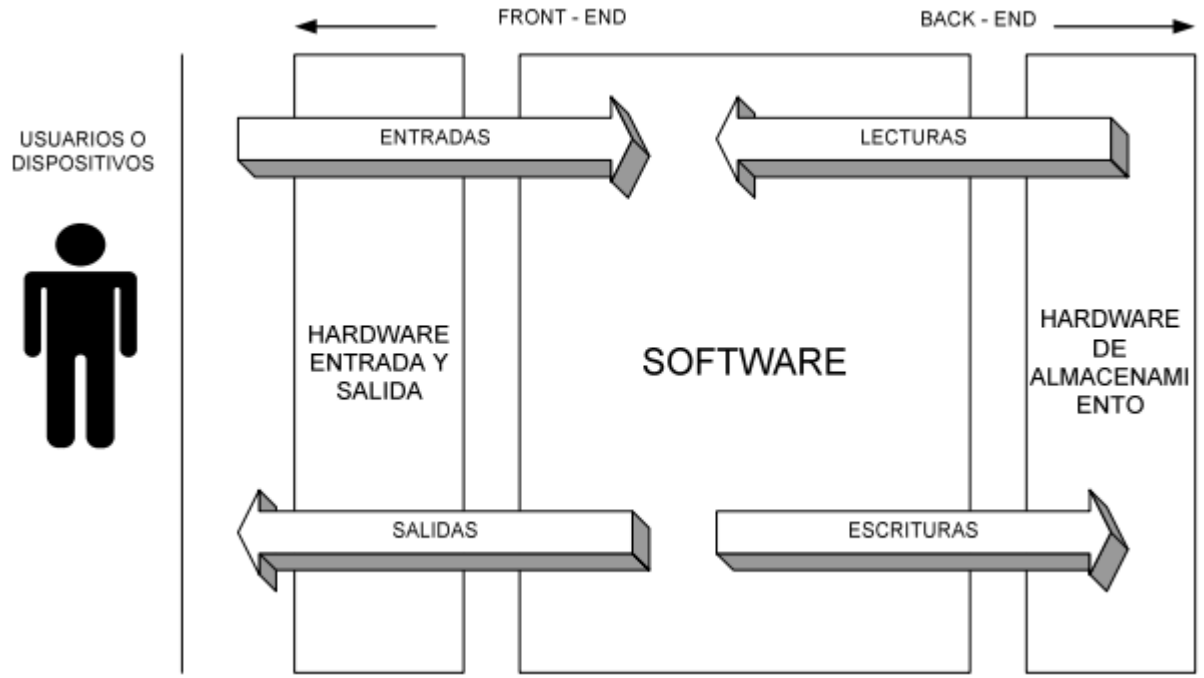
Fase II: La fase de mapeo de FURs al modelo conceptual de COSMIC

- Algunos **principios** de los conceptos de COSMIC son:
  1. El software está estructurado por **capas**
  2. El software ejecuta **funciones** con base en eventos disparados por los usuarios
  3. Cada **único evento** dispara un único proceso funcional, el mismo que está completo cuando se ha generado la respuesta al evento
  4. Un proceso funcional comprende **subprocesos** que mueven o manipulan datos.

Fase II: La fase de mapeo de FURs al modelo conceptual de COSMIC

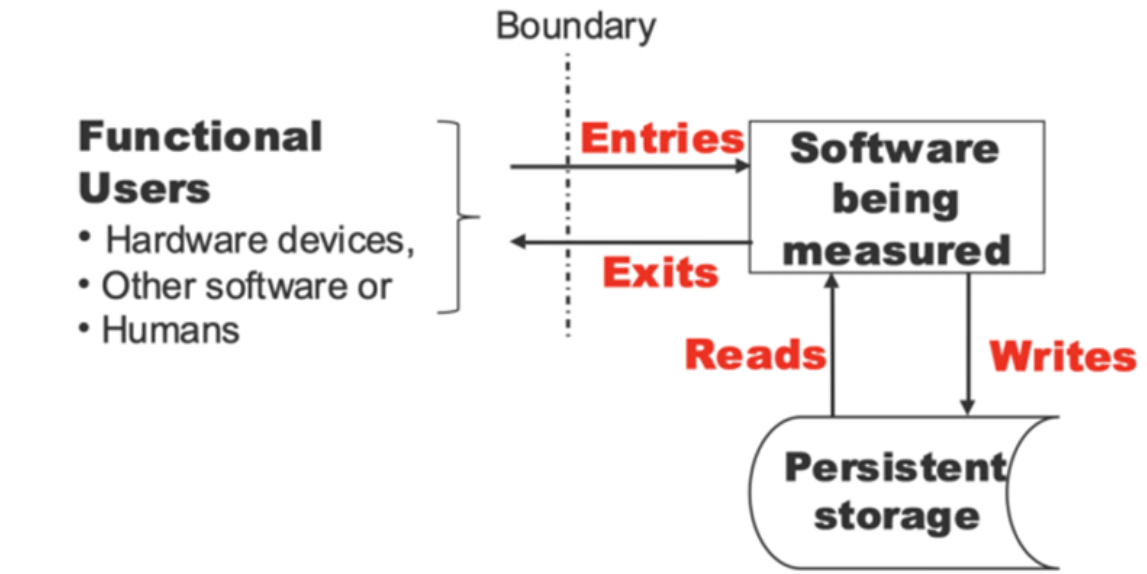
5. Un **movimiento de datos** es el movimiento de un único grupo de datos
6. Contempla cuatro subprocesos de movimientos de datos:
   - ❖**Entrada**: transfiere datos desde los usuarios hacia el software
   - ❖**Salida**: transfiere datos desde el software hacia los usuarios
   - ❖**Lectura**: transfiere datos desde el almacenamiento
   - ❖**Escritura**: transfiere datos hacia el almacenamiento

Fase II: La fase de mapeo de FURs al modelo conceptual de COSMIC



Flujo genérico de datos en COSMIC
Fuente: http://www.itba.edu.ar

Fase II: La fase de mapeo de FURs al modelo conceptual de COSMIC

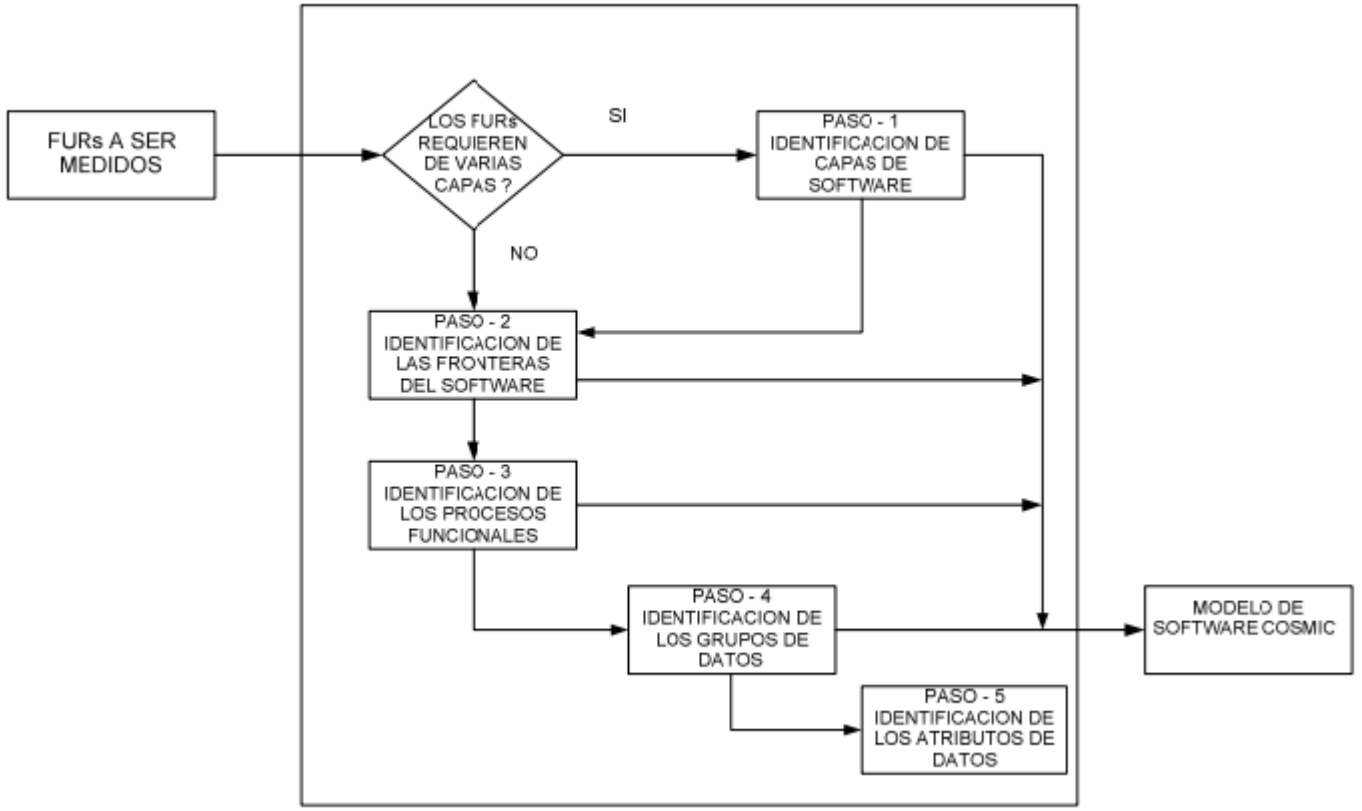


The 'Data Movement' is the unit of measure: 1 CFP

Copyright © COSMIC 2017

Fuente: https://cosmic-sizing.org/cosmic-fsm/

Fase II: La fase de mapeo de FURs al modelo conceptual de COSMIC



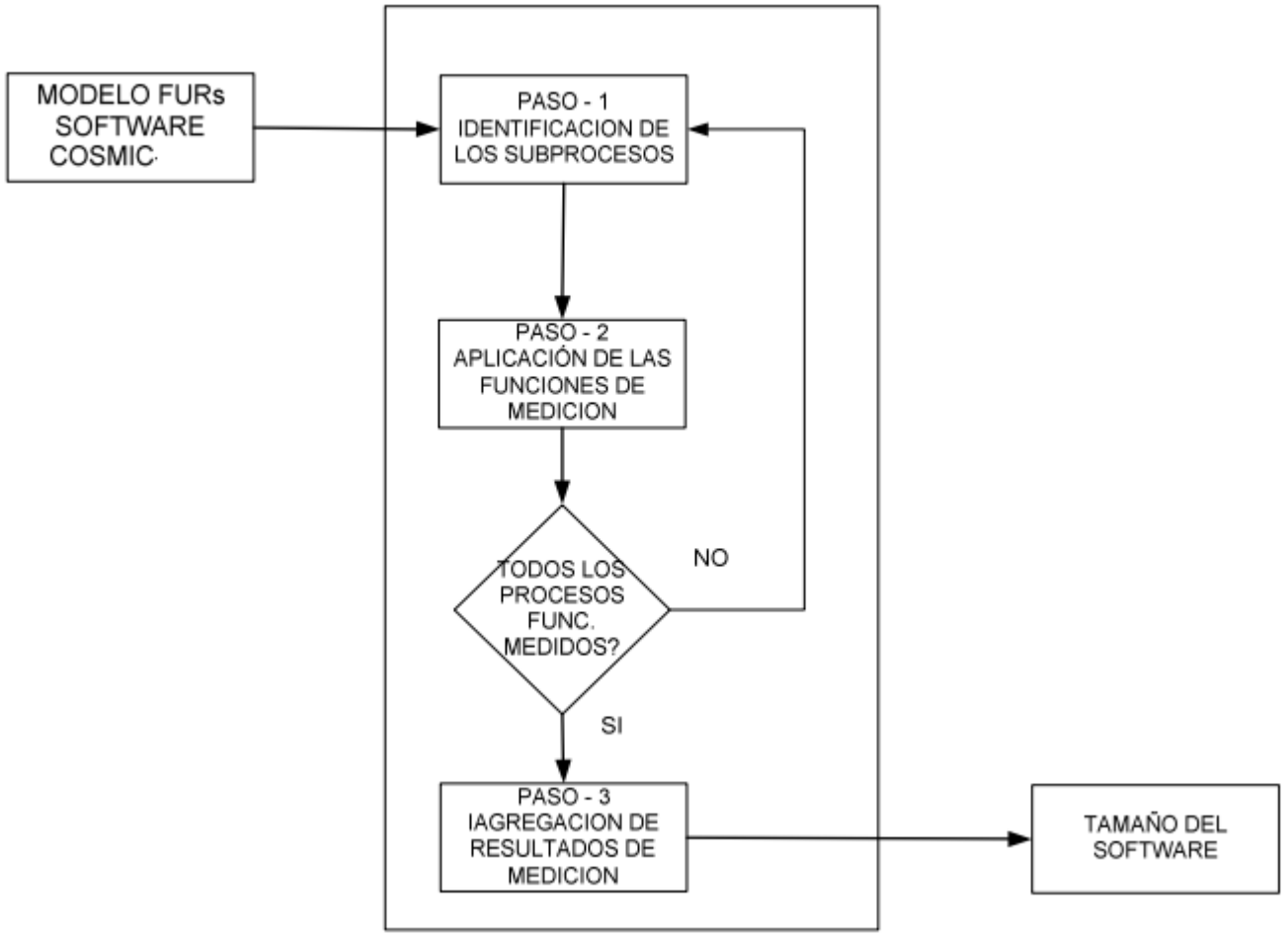
Modelo general para el proceso de representación COSMIC

Fuente: http://www.itba.edu.ar

Fase III: La fase de medición de COSMIC

- Toma como entrada una **instancia del modelo de software COSMIC**
- Con base en **reglas y procedimientos** genera el **valor** que es directamente proporcional al tamaño del software en medición. Este valor es expresado en **CFP** [COSMIC Function Point]

Fase III: La fase de medición de COSMIC



Procedimiento general de la fase de medición
Fuente: http://www.itba.edu.ar

Fase III: La fase de medición de COSMIC

- Identificar los **movimientos de datos**
- Aplicar la **función de medición**
- **Agregar los resultados** de la medición con base en las reglas de agregación:

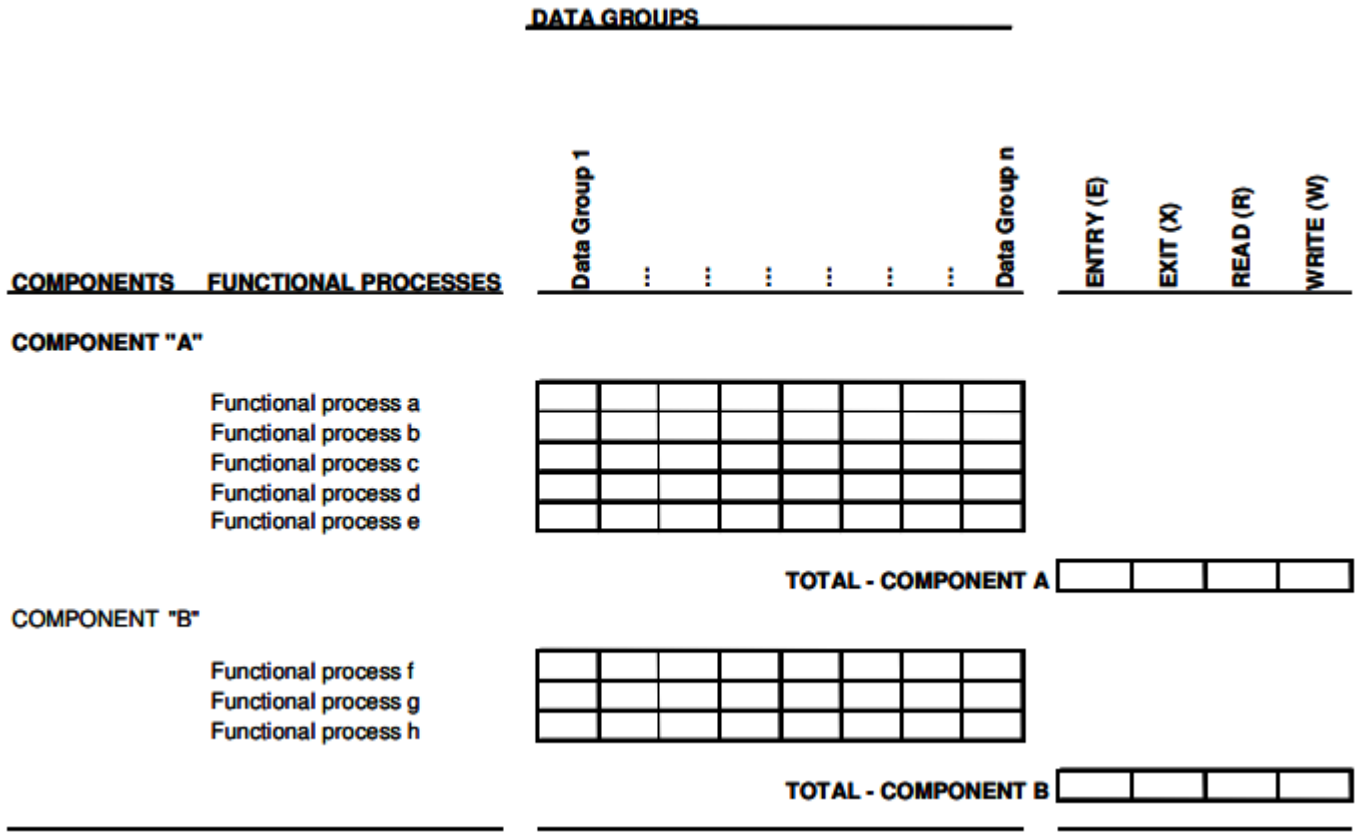
$$\text{Size (functional process}_i\text{)} = \Sigma \text{ size(Entries}_i\text{)} + \Sigma \text{ size(Exits}_i\text{)} + \Sigma \text{ size(Reads}_i\text{)} + \Sigma \text{ size(Writes}_i\text{)}$$

$$\text{Size (Change(functional process}_i\text{))} = \Sigma \text{ size (added data movements}_i\text{)} + \Sigma \text{ size (modified data movements}_i\text{)} + \Sigma \text{ size (deleted data movements}_i\text{)}$$

Fase IV: La fase del reporte de la medición

- La medición es presentada en una matriz en donde las **filas representan los procesos funcionales** agrupados por capas, las **columnas representan los grupos de datos** y las celdas identifican los subprocesos: Entrada, Salida, Lectura, Escritura.

Fase IV: La fase del reporte de la medición



Matriz de medición del tamaño de un software con COSMIC
Fuente: http://www.cosmicon.com

Beneficios del uso de COSMIC

- **Compatible** con los métodos modernos para documentación de requerimientos de software.
- **Exige** realizar adecuadamente el proceso de levantamiento de requerimientos de software
- Es fácil de aprender y tiene un adecuado **balance costo-beneficio**
- Es posible **automatizar** el proceso de **dimensionamiento** de los requerimientos que son manejados en herramientas CASE
- Preciso **independientemente del tamaño** del software

Limitaciones del uso de COSMIC

- **No** estima el esfuerzo en requerimientos **no funcionales** del software
- **Excluye** los componentes físicos o técnicos relacionados con el hardware.

Next On

Take Away Points

- The input to the COSMIC estimation method is the functional requirements of the software.
- COSMIC is applicable for business software and real-time software.
- COSMIC depends on the quality of the requirements gathering process.
- COSMIC does not estimate the effort on technical or quality requirements.

Further Reading

- Laird and Brennan, “Software Measurement and Estimation: A Practical Approach”.
  - Chapters 4, 5, and 6.
- Trendowicz and Jeffery, “Software Project Effort Estimation: Foundations and Best Practice Guidelines for Success”.
  - Preface, Chapters 2, 3 and 4.
- Steve McConnell, “Software Estimation: Demystifying the Black Art”.
  - Chapters 1 - 13
- Peter Hill, “Practical Software Project Estimation: A Toolkit for Estimating Software Development Effort & Duration”.
  - Chapters 19, 20, 21
- COSMIC, “The Standard Methodology for Sizing Software”
  - https://cosmic-sizing.org
- ISO/IEC 19761:2011, “Software engineering - COSMIC: a functional size measurement method”.
- B13.ai, “De-Risking Software Development: White Paper”

Next Lecture

- Risk Management